

University Enrollment System

A Deep Dive into a PyQt6 + SQLAlchemy Course Registration Desktop App

Explainer document for Salman Adnan

Read this first: what this document is

This is an exhaustive, beginner-safe walkthrough of the **university-enrollment-system** project. It assumes no prior knowledge: every technical term is defined at first use.

It is a working document for your own understanding and interview defence, not marketing copy. Where the code is weak, wrong, or disagrees with the README, this document says so plainly, in a red box. A flaw you learn about here is a flaw you can defend in a room. A flaw an interviewer finds first is not.

Two sections carry the weight of the project and are treated in full depth: the **data model** (Section 3) and the **enrollment rules** (Section 4).

Contents

1	What this project is	3
1.1	The one-sentence version	3
1.2	Provenance and scope	3
1.3	How the facts in this document were established	3
2	Orientation: the shape of the whole thing	3
2.1	The file layout	5
3	The data model	6
3.1	The five tables, drawn	7
3.2	Table by table	7
3.3	The foreign keys do not work, and cannot	10
3.4	A relationship that is quietly mis-declared	11
3.5	The import-time side effects	11
3.6	The dead model file	12
4	The enrollment rules	12
4.1	The cart: a two-phase commit, done by hand	12
4.2	Rule 1: can this course be added?	13
4.3	Rule 2: the clash detector	15
4.4	Rule 3: capacity, and what <code>current_capacity</code> actually means	17
4.5	Rule 4: the swap	18
4.6	Rule 5: the drop	20
4.7	The enroll screen, and a bug that cancels itself out	20
5	The data-access layer	21
5.1	<code>glob_id</code> : how the layer knows who you are	21
5.2	Error handling, uniformly	22
5.3	Two methods that do the same job	23
5.4	The destructive <code>__main__</code> block	23
6	Login and the verification flow	24

7	The remaining screens	27
7.1	Attendance	27
7.2	View schedule and the Excel export	27
7.3	Activity log	29
7.4	The window-transition machinery	29
7.5	The name collision	30
8	Packaging and running	30
8.1	Dependencies, and what each is for	31
9	The verification story, and what it does not cover	32
10	Findings, ranked	32

1 What this project is

1.1 The one-sentence version

A desktop application that lets a university student log in, receive a code by email, confirm it, and then build a course timetable: browse a catalogue of course sections, put some in a cart, enroll, drop, swap sections, look at the resulting weekly schedule, export it to a spreadsheet, mark attendance, and read a log of everything they did.

1.2 Provenance and scope

Per the project README, this was the semester project for the Databases course (CS/CE 355/373) at Habib University, built by a two-person team, Salman Adnan and Shayan Wasif, in the 4th semester. The README records Salman's areas as the email verification (login MFA) flow, the attendance module with its email confirmations, the Docker packaging, and integration plus bug fixing across the screens.

Jargon: README

A **README** is the plain-text introduction file that sits at the root of a code repository. By convention it is the first thing a reader opens. It is written by humans and is not executed, so it can drift out of step with the code. Throughout this document, where the README and the code disagree, **the code wins**, and the disagreement is recorded as a finding.

By size this is a mid-weight project: 13 Python modules totalling roughly 2,570 lines, 11 interface-definition files, and one pre-seeded database file. That is small enough to cover completely, which this document does, and large enough that the structure matters.

1.3 How the facts in this document were established

This matters, because it tells you which claims you can defend hard and which you cannot.

- **Read.** Every tracked file was read end to end.
- **Queried.** The shipped `student.db` was opened read-only and its schema, row counts, and actual data were inspected directly. Every number quoted about the database in this document came out of the file, not out of the README.
- **Executed.** The data-access layer (`queries.py`) was imported into a throwaway virtual environment and run against a *copy* of the database, to test the capacity arithmetic and the swap operation. The results are reported in Sections 4.4 and 4.5. The original database was not touched.
- **Executed.** PyQt6 was installed in that same throwaway environment purely to settle one question of fact about enum truthiness, in Section 4.7.
- **Not executed.** The graphical application itself was never launched: PyQt6 is not installed on the machine this document was written on, and the app needs a desktop. Any claim about on-screen behaviour is therefore a reading of the code, and is labelled as such.
- **Not executed.** The Docker image was never built: the Docker daemon was not running. Section 8 contains one clearly labelled inference as a result.

2 Orientation: the shape of the whole thing

Before any detail, here is the entire application in one picture. Every box is a window the user can be looking at; every arrow is a transition the code actually wires up.

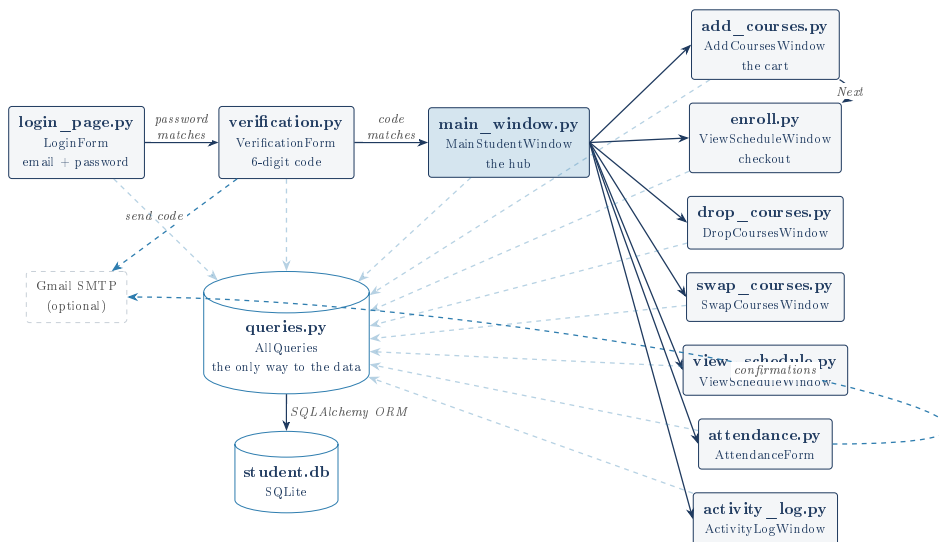


Figure 1: The whole application. Solid arrows are window transitions; faint dashed arrows are data-access calls. Every screen without exception reaches the database through **AllQueries**. That single choke point is the best structural decision in the project.

Jargon: Desktop app, GUI, and PyQt6

A **GUI** (Graphical User Interface) is software you click on, as opposed to one you type commands at. **Qt** is a long-established C++ toolkit for building GUIs: it supplies the buttons, tables, and windows. **PyQt6** is a Python binding for Qt version 6, meaning a translation layer that lets Python code drive Qt's C++ machinery. This project is a **desktop app**: it runs as a program on your own computer and draws its own windows, unlike a web app, which runs in a browser.

Jargon: SQLite

SQLite is a database that lives in a single ordinary file on disk, with no server process to install or start. Here that file is `student.db`. This makes it ideal for coursework and for shipping a demo, since a clone of the repository comes with its data already in it, and less appropriate for anything with many simultaneous users.

Jargon: ORM and SQLAlchemy

A database stores rows in tables. Python works in objects. An **ORM** (Object-Relational Mapper) is the layer that translates between the two, so that a row in the `student` table appears in Python as a `Student` object with a `.name` attribute. **SQLAlchemy** is the standard Python ORM. It means you rarely write SQL by hand; you write `session.query(Student).filter_by(std_id=123).first()` and SQLAlchemy emits the `SELECT` for you.

2.1 The file layout

```

university-enrollment-system/
├── login_page.py    entry point
├── verification.py  MFA code screen
├── main_window.py   hub
├── add_courses.py   the cart
├── enroll.py        checkout
├── drop_courses.py
├── swap_courses.py
├── view_schedule.py  + Excel export
├── attendance.py
├── activity_log.py
├── queries.py        data-access layer
├── student_database.py  ORM models + engine
├── courses_database.py  DEAD, never imported
├── *.ui             11 Qt Designer layouts
├── student.db        pre-seeded SQLite file
├── requirements.txt
├── Dockerfile
├── run-linux.sh, run-macos.sh, run-windows.ps1
├── .env.example
├── Proposal.pdf
├── docs/
│   ├── screenshots/app.png
│   └── explainers/    this document

```

Figure 2: Every tracked file. The project is flat: no packages, no `src/` directory, one module per screen.

The flat layout is a real constraint, not just a style choice

Every screen calls `uic.loadUi('name.ui', self)` with a *relative* path. A relative path is resolved against the process's current working directory, not against the location of the Python file. The practical consequence: the application only runs if you launch it from inside the project directory. Run `python /some/where/login_page.py` from your home directory and it crashes on the first `loadUi`. The README states this plainly and it is why the Dockerfile sets `WORKDIR /app`. It is worth being able to say out loud: this is a bug that packaging has to work around rather than a decision.

Jargon: Qt Designer and .ui files

Rather than writing code to place every button, you can draw the window in a visual editor called **Qt Designer**, which saves the layout as a `.ui` file (an XML document listing widgets, their names, and their positions). `uic.loadUi` reads that file *at runtime* and builds the widgets, attaching each one to the window object under its Designer name. That is why `self.login_button` exists in `login_page.py` even though no line of Python ever created it.

3 The data model

This is a Databases course project, so the schema is the centre of gravity. Everything in this section was read out of `student_database.py` and then verified against the actual `student.db` file.

Jargon: Schema, table, row, column

A **table** is a grid of data, like a spreadsheet tab. A **column** is one named field with a type (a whole number, a string of text, a time). A **row** is one record. The **schema** is the full description of which tables exist and what columns they have. In SQLAlchemy the schema is written as Python classes, and `Base.metadata.create_all(engine)` turns those classes into real tables.

Jargon: Primary key and foreign key

A **primary key** (PK) is the column, or the combination of columns, that uniquely identifies a row. No two rows may share it. When a PK is made of more than one column it is called a **composite** primary key. A **foreign key** (FK) is a column that points at another table's primary key, expressing a relationship: "this enrollment row belongs to that student". A database can *enforce* foreign keys, refusing to store a row that points at a student who does not exist. Whether this one does is the subject of Section 3.3, and the answer will surprise you.

3.1 The five tables, drawn

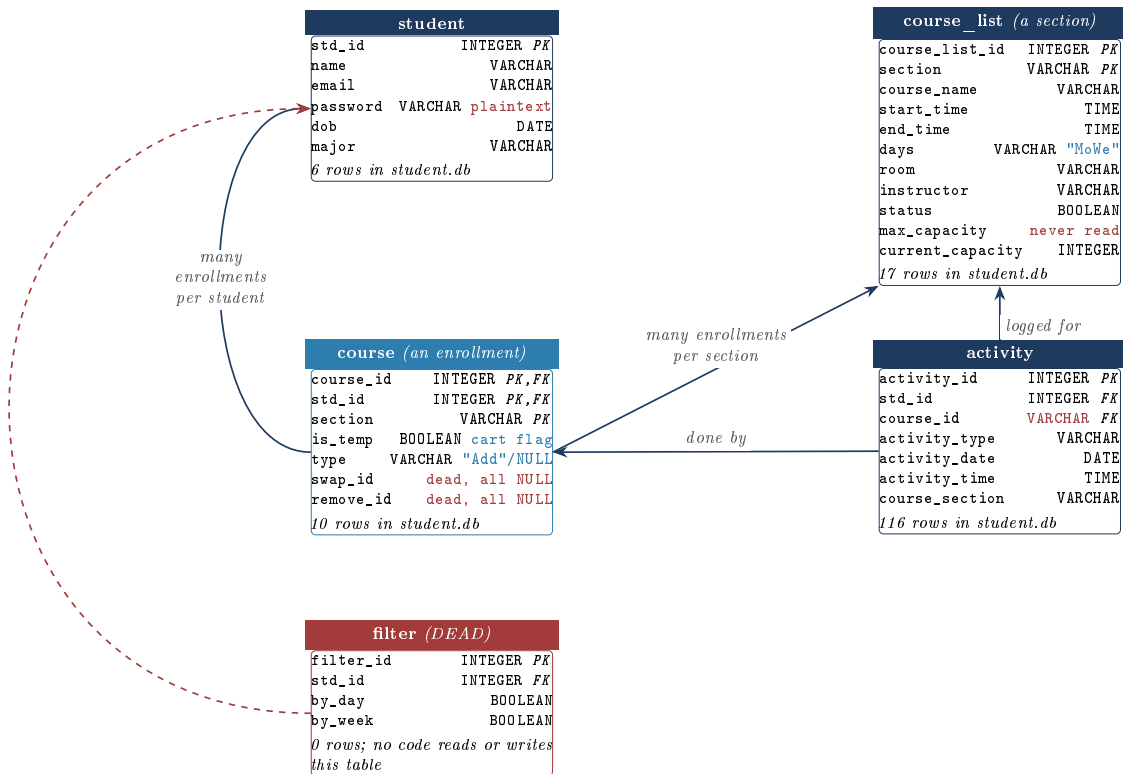


Figure 3: The full schema as it exists in `student.db`, with row counts taken from the shipped file. Blue is the junction table that carries the whole enrollment idea. Red marks what is declared but dead.

3.2 Table by table

3.2.1 student

Six columns, one per student, keyed by `std_id`. The shipped database holds 6 rows: Alice (20180), Bob (123), Steve (153), Babar (173), Umer (257), Ronaldo (273).

Passwords are stored in plaintext, and the README says so

`Student.password` is a plain `String`. `login_page.py` fetches it with `get_password_by_email(email)` and compares with `password == user_password`, a direct string comparison. There is no hashing, no salt, nothing.

This is genuinely bad, and the right way to talk about it is exactly how the README already does: it is a coursework limitation, it is called out in the Usage section, and hashing with `bcrypt` is listed as a follow-up. Do not defend it as a design choice. Say: “it is plaintext, I know why that is wrong, I documented it rather than hiding it, and the fix is `bcrypt` at the two call sites.”

3.2.2 course_list: the catalogue

This is the table of *sections*, not of courses. The distinction is the single most important thing to understand about this schema.

A section, not a course, is the unit of everything

`course_list` has a **composite primary key**: (`course_list_id`, `section`). So “Data Structure II” is not one row. It is three:

id	section	name	days	start	end
201	L1	Data Structure II	MoWe	11:30	12:45
201	L2	Data Structure II	WeFr	14:30	15:45
201	L3	Data Structure II	MoWe	16:00	17:15

The course *name*, and every timing detail, is repeated on each section row. A student enrolls in a section. A clash is between sections. A swap, as Section 4.5 shows, is defined as moving between two sections of the *same* `course_list_id`. Once you see that “a row is a section”, the rest of the schema reads cleanly.

The catalogue is not normalised, and this is a Databases project

Because the course name lives on every section row, “Data Structure II” is stored three times. Rename the course and you must update three rows or the catalogue disagrees with itself. The textbook fix is two tables: a `course` table (`course_id`, `course_name`) and a `section` table referencing it with the timing, room, instructor, and capacity.

This is worth rehearsing precisely because of the course it was submitted to. An examiner looking for normalisation will find this immediately. The honest answer is that the denormalisation is real, it buys nothing here except simpler queries, and the two-table split is what you would do now.

The `days` column deserves a note: it is a string of two-character day codes glued together, so “MoWe” means Monday and Wednesday. The whole catalogue uses only four patterns, verified against the file: MoWe, WeFr, TuTh, and one MoSa. Section 4.3 shows how the clash checker takes this string apart.

Junk in the seeded catalogue

Row (520, 'S1') is called “Test Course”, runs 02:30 to 02:45 on MoSa, and is plainly leftover test data that shipped. It is 15 minutes long, at half past two in the morning, on Monday and Saturday. If you demo the app live, this row will appear in the catalogue. Either delete it from the seed or be ready to explain it.

3.2.3 course: the junction table, and the cart

Jargon: Junction table (also: join table, associative entity)

A student takes many sections; a section holds many students. That is a **many-to-many** relationship, and a relational database cannot express it with a single foreign key on either side. The standard solution is a third table whose rows each pair one student with one section. That is a **junction table**. Here it is `course`, and its primary key (`course_id`, `std_id`, `section`) is exactly the statement “this student holds this section of this course, at most once”.

The clever part is that this table does double duty. It holds both what you are *enrolled in* and what is sitting in your *cart*, distinguished by two columns:

is_temp	type	meaning
True	"Add"	in the cart, not yet enrolled
False	NULL	really enrolled

Verified against the shipped database: every one of the 10 `course` rows has `is_temp` = 0 and `type` = NULL. That is, the shipped state has nobody with anything in their cart, which is what you would expect from a clean save.

swap_id and remove_id are dead columns

Both are declared on the `Course` model, both have foreign keys, and the source even carries a commented-out earlier version of their definitions. No code anywhere assigns either one. Verified: all 10 rows have both NULL. They are the fossil of a design where a swap was going to be recorded as a link between two `course` rows rather than as a delete plus an insert. Section 4.5 shows what was built instead, and why that fossil is a bit of a shame.

3.2.4 activity: the log

Every action appends a row here. The shipped database has 116 of them. The distinct `activity_type` values, read out of the file, are exactly: `Add`, `Enroll`, `Drop`, `Swap`, `Filter by Day`, `Filter by Week`, `Export Schedule`, `Attendance`.

activity.course_id is a VARCHAR that stores prose

The column is declared `Column(String, ForeignKey('course_list.course_list_id'))`. It is a *text* column claiming to be a foreign key to an *integer* primary key. It is not used as an identifier at all. These are real, distinct values read out of the shipped table:

```
'201'      '201, 202'      '201, 202, 205, 310, 330'      '201 -> 201'      'None'      NULL
```

So the column variously holds one id, a comma-joined list of ids from a multi-course enroll, an arrow expression from a swap, the four-character string `'None'`, and a genuine SQL NULL. `course_section` is the same story: it holds `'L1'`, `'L1, S1'`, `'L3 -> L2'`, and for attendance rows an entire sentence.

The log is therefore a **denormalised, human-readable audit trail**, not a queryable relation. You cannot ask “how many times was course 201 dropped” without parsing strings. The honest framing: it was built to be rendered into a table widget, and it does that job, but calling it a foreign key in the model is a false claim the schema makes about itself.

Note the `'None'` strings specifically: current code writes `str(course_id)` if `course_id` else `None`, which yields a real NULL for a missing id. So those `'None'` rows were written by an *earlier* version of the code that called `str()` unconditionally. The shipped database is a historical artifact spanning more than one generation of this function. That is a fine thing to notice out loud; it shows you read the data and not just the code.

3.2.5 filter: declared, created, never used

The filter table is entirely dead

`Filter` is a full SQLAlchemy model with `by_day` and `by_week` booleans and a relationship back to `Student`. `create_all` duly creates the table in SQLite. Then: no module imports it, no query touches it, and it holds **0 rows** in the shipped database. Verified both ways, by `grep` and by row count.

What happened is visible in `view_schedule.py`: the day/week toggle state ended up living in a plain Python attribute, `self.filter_mode`, which is set to "Day" at construction and flipped by the buttons. It is never persisted. The `filter` table is what persisting it would have looked like. The screen instead records each toggle as an `activity` row of type `Filter` by `Day`, which is logging, not state.

3.3 The foreign keys do not work, and cannot

This is the most substantial finding in the schema, and it was established by running SQLite against the shipped file, not by reading.

SQLite does not enforce foreign keys unless you ask it to

Almost every other database enforces foreign keys by default. SQLite does not: for backward compatibility, foreign-key enforcement is **off** unless the connection issues `PRAGMA foreign_keys = ON`. Verified on the shipped file: the pragma reads 0. Nothing in `student_database.py` or `queries.py` turns it on. So every `ForeignKey` in the model is, at runtime, pure documentation.

Demonstrated directly against a copy of the database, with enforcement at its default:

```
1 INSERT INTO course VALUES (99999, 88888, 'ZZ', 0, NULL, NULL, NULL);
2 -- ACCEPTED. No such course_list_id 99999. No such std_id 88888.
```

Listing 1: A row referencing a student who does not exist and a course that does not exist. Accepted without complaint.

Now the twist. You might think the fix is one line: turn the pragma on. It is not, because the foreign keys as declared are *structurally invalid* and SQLite will refuse them:

```
1 PRAGMA foreign_keys = ON;
2 PRAGMA foreign_key_check;
3 -- OperationalError: foreign key mismatch - "activity" referencing "course_list"
4
5 INSERT INTO course VALUES (99999, 88888, 'ZZ', 0, NULL, NULL, NULL);
6 -- OperationalError: foreign key mismatch - "course" referencing "course"
```

Listing 2: With enforcement enabled, the schema itself errors.

Why the foreign keys are invalid: they reference half of a composite key

A foreign key must point at a set of columns that is unique in the parent table. Two of these do not:

- `course.course_id` and `activity.course_id` both reference `course_list.course_list_id`. But `course_list`'s primary key is the *pair* (`course_list_id`, `section`). `course_list_id` alone is **not unique**: id 201 appears three times, once per section. So the FK points at something that does not identify a row, and SQLite calls it a mismatch. The correct declaration is a composite FK on (`course_id`, `section`).
- `course.swap_id` and `course.remove_id` reference `course.course_id`, which is likewise only one third of `course`'s composite PK (`course_id`, `std_id`, `section`). Same mismatch. The commented-out lines just above them in the source show the author already wrestling with which table these should point at.

The two facts compose into the finding: **the schema's referential integrity is broken, and the only reason nothing has ever failed is that SQLite was never asked to check it.** Turn the pragma on and the application stops working. For

a Databases course project this is the thing to be able to explain, and it is entirely explicable: composite primary keys make foreign keys harder, SQLAlchemy will happily emit an FK clause it cannot validate, and SQLite’s silence removed the feedback that would have caught it. The fix is `ForeignKeyConstraint(['course_id', 'section'], ['course_list.course_list_id', 'course_list.section'])` declared at the table level instead of two column-level `ForeignKey` calls.

3.4 A relationship that is quietly mis-declared

Course.course_list is declared one-to-one and is not

`student_database.py` declares:

```
1 course_list = relationship('CourseList', foreign_keys=[course_id],
2                             back_populates='course')
```

with the source comment “Define one-to-one relationship between `Course` and `CourseList`”. Because it joins on `course_id` alone and ignores `section`, a `Course` row for course 201 matches all *three* `course_list` rows for 201. This is not theoretical. Running the query layer surfaced it immediately as a real SQLAlchemy warning:

```
1 SAWarning: Multiple rows returned with uselist=False for lazily-loaded
2 attribute 'Course.course_list'
```

The application never trips over this because no screen uses the relationship. Every screen instead calls `get_course_by_id_and_section(course_id, section)`, which does the join by hand with both columns and gets the right answer. So the ORM relationship is dead weight that is also wrong, and the manual query beside it is correct. That is the same root cause as Section 3.3: half of the composite key was dropped.

3.5 The import-time side effects

```
1 engine = create_engine('sqlite:///student.db', echo=True)
2 ...
3 Base.metadata.create_all(engine)
4 Session = sessionmaker(bind=engine)
5 session = Session()
6 session.close()
```

Listing 3: The tail of `student_database.py`. All of this runs on import.

Importing any screen creates an engine, writes the schema, and spams SQL

Four things happen the moment *anything* does `import queries`, because `queries.py` imports `student_database`:

1. An engine is created against `student.db`, bound to the current working directory.
2. `echo=True` means SQLAlchemy prints **every SQL statement it ever emits** to the terminal, forever, for the life of the process.
3. `create_all(engine)` runs, touching the database file and creating any missing table.
4. A module-global `session` is opened, then immediately `close()`d.

The README already flags this. Two extra things worth knowing that it does not say. First, the `session.close()` on the last line looks like a bug and is not, quite: SQLAlchemy sessions re-open a connection lazily on next use, so the module-global `session` that every

query in `queries.py` uses still works. It is confusing and pointless, but harmless. Second, and more seriously: **there is exactly one session for the whole process**, shared by every screen. It is never committed or rolled back on any boundary the user can see. Combined with the non-atomic swap in Section 4.5, that shared session is the thing that makes a half-finished operation stick.

3.6 The dead model file

`courses_database.py` defines a second, incompatible `Student`

The README describes `courses_database.py` as “a seeding helper for the course catalog”. **Read the file: it is not.** It contains no course data and no seeding code at all. What it contains is a second SQLAlchemy `Base`, a second engine against the same `student.db`, and a second `Student` model whose primary key is called `id` (not `std_id`) and whose `dob` is a `String` (not a `Date`). Both of those contradict the real `student` table. Everything else in the file is commented out.

Verified: `grep` finds no module that imports `courses_database`. It is dead, its `create_all` is commented out (which is the only reason it has never corrupted anything), and the README’s one-line description of it is wrong. This file should be deleted. If an interviewer opens it expecting the seeding helper the README promised, you want to be the one who already knows.

4 The enrollment rules

This is the functional heart of the project. There are four operations the student can perform on their timetable, and each has its own rules, enforced at different layers and with different degrees of rigour.

4.1 The cart: a two-phase commit, done by hand

Adding is not enrolling

The project separates **choosing** from **committing**, exactly like an online shop. `add_courses.py` is the cart: picking a course writes a `course` row with `is_temp=True`, `type="Add"`. Nothing about the catalogue changes; no seat is taken. Only when the student goes to `enroll.py` and presses Enroll does the flag flip to `is_temp=False`, `type=NULL` and the seat count move.

The value of the split is that a cart can be abandoned. The cost is that a `course` row means two different things depending on a boolean, and every query in `queries.py` has to remember to filter on it. Several do; one, as we will see, does not.

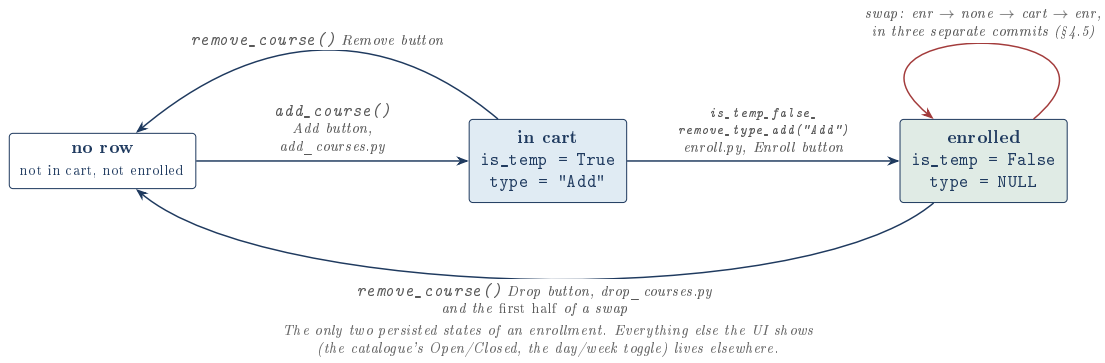


Figure 4: The lifecycle of a course row. The red self-loop is the swap, and it is the one transition that is not safe.

4.2 Rule 1: can this course be added?

The gate is `AddCoursesWindow.can_the_course_be_added(course_id, start_time, end_time, days)`, backed by a second check inside `AllQueries.add_course`.

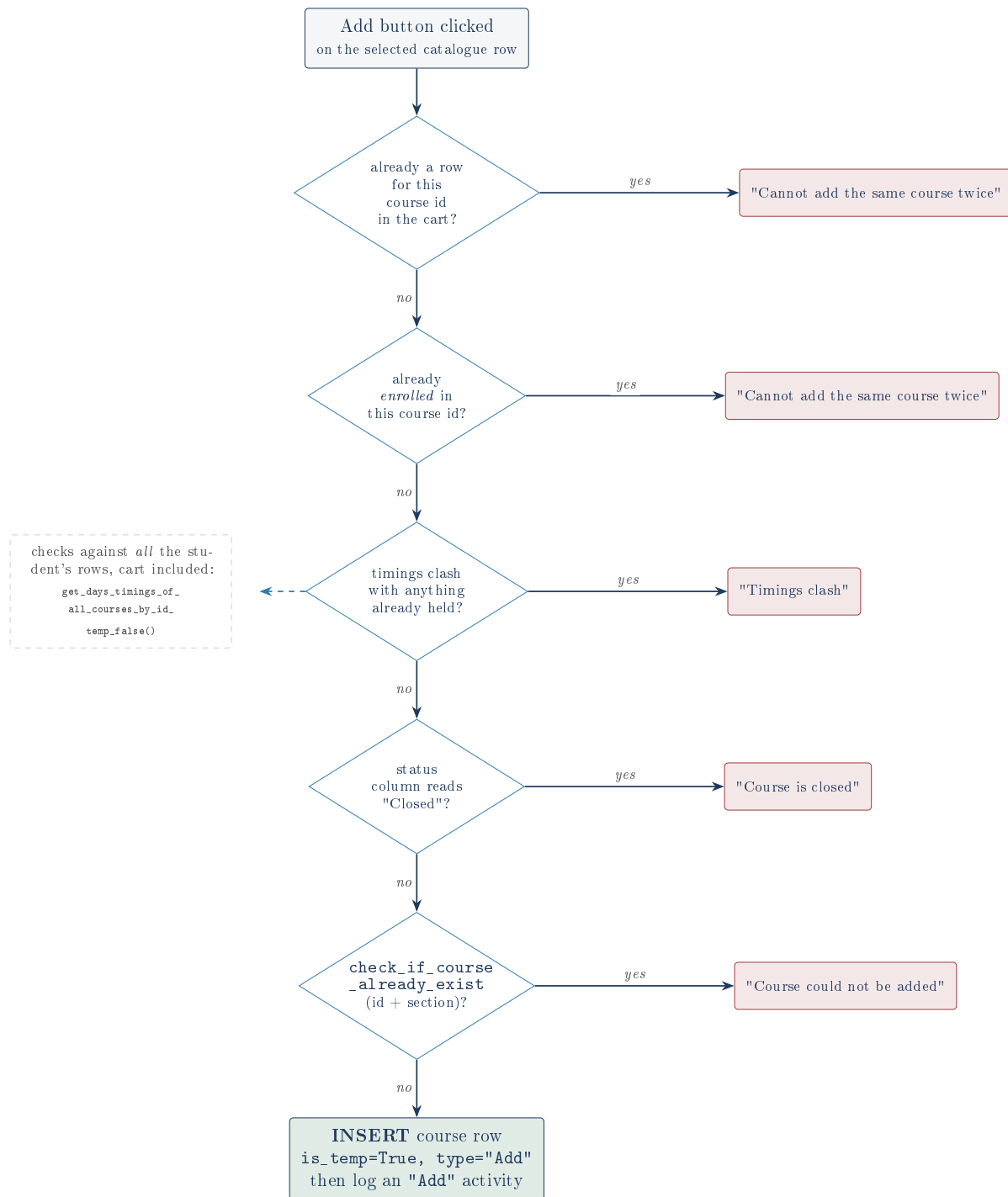


Figure 5: The add gate, in the order the code actually evaluates it. Note the first four tests live in the window class and only the last lives in the query layer.

Three things about that flowchart are worth stopping on.

The “already added” rule is by course id, so you cannot take two sections of one course, and the duplicate check disagrees with itself

The two top gates compare **course id only**, ignoring section. So having 201/L1 in your cart blocks you from adding 201/L2. That is almost certainly the intent (you take one section of a course), and the message “Cannot add the same course twice” says so.

But the last gate, `check_if_course_already_exist(course_id, section)` inside

`AllQueries.add_course`, compares **id and section together**. So the query layer would happily allow 201/L1 and 201/L2 to coexist; only the UI stops it. And the database agrees with the query layer, because `course`'s primary key includes `section`.

So the rule “one section per course” is enforced in exactly one place, the weakest one: a window class. Nothing in the schema or the data layer encodes it. Anything that inserts by another path, including `swap_courses`, bypasses it entirely.

`get_days_timings_of_all_courses_by_id_temp_false` does not filter on `is_temp`

Read the name, then read the body:

```
1 def get_days_timings_of_all_courses_by_id_temp_false(self):
2     std_id = self.glob_id
3     course_list_query = session.query(Course).filter_by(std_id=std_id).all()
```

No `is_temp=False`. Its sibling one method above, `get_timings_of_all_courses_by_id`, does filter. So the method whose name promises to exclude cart rows is the one that includes them.

Here is the subtle part, and it is worth getting right rather than just calling it a bug: **the unfiltered behaviour is the behaviour the cart actually needs**. When you are adding a third course, you want it checked against your enrolled courses *and* the two already in your cart. If it filtered as its name promises, you could fill a cart with three mutually clashing courses and enroll in all of them. So the code is right and the name is a lie. That is a much better thing to say in an interview than “there is a bug”. The fix is a rename, to something like `get_days_timings_of_all_courses_by_id`, and a comment saying why the cart counts.

The clash check runs before the closed check, and both run before either matters

In `add_course`, `can_the_course_be_added(...)` is called on line 216 and its result is stored, then the `status == "Closed"` test runs on line 219 and returns first. So on a closed course that also clashes, the user sees the “Timings clash” warning popup (raised from inside the clash check) and *then* the “Course is closed” popup. Two dialogs for one click. Cosmetic, but it is the kind of thing a demo surfaces.

4.3 Rule 2: the clash detector

This is the most algorithmically interesting code in the project. It answers: does the candidate section overlap in time, on a shared day, with anything the student already holds?

Jargon: Interval overlap

Two time intervals overlap if one starts before the other ends and ends after the other starts. The naive way to test it is to enumerate cases: does A's start fall inside B? does A's end fall inside B? does A completely contain B? Those three cases are exhaustive, and they are exactly the three conditions this code tests.

The code works in two steps: reduce times to a comparable number, then compare.

```
1 def time_to_seconds(self, t):
2     return t.hour * 3600 + t.minute * 60 + t.second
3
4 def get_days_list(self, days):
5     return [days[x:x + 2] for x in range(0, len(days), 2)]
```

Listing 4: `queries.py`: times become integers.

`get_days_list` is how "MoWe" becomes ['Mo', 'We']: it slices the string into two-character chunks. This is the entire day-encoding scheme, and it is why the catalogue must never contain a day code of any other length.

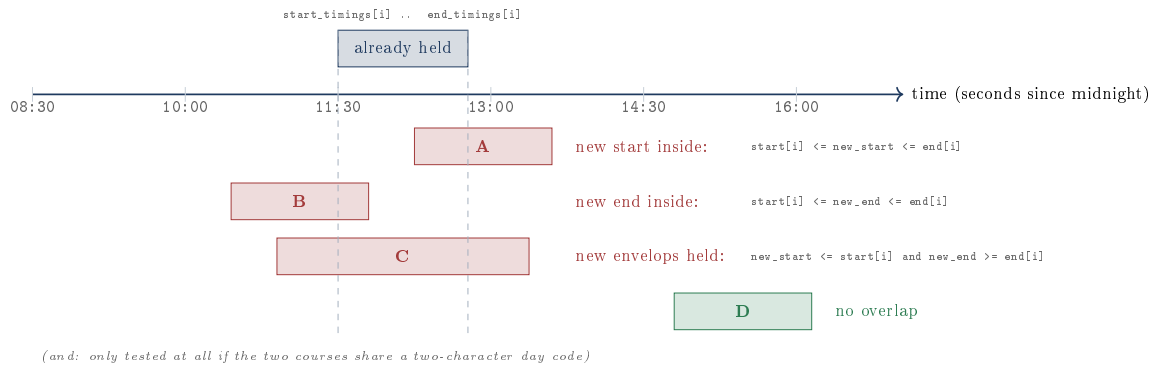


Figure 6: The three overlap cases the clash detector tests, drawn against one already-held section. A, B and C together are exhaustive: any overlap of two intervals falls into at least one of them.

The clash rule, as implemented in `can_the_course_be_added`

```
new_start, new_end := time_to_seconds(candidate start, end)
for each course the student already holds (enrolled OR in cart), as i:
    shared := any 2-char day code of the candidate
               appears in the day codes of course i
    if shared:
        if start[i] <= new_start <= end[i] -> CLASH
        if start[i] <= new_end <= end[i] -> CLASH
        if new_start <= start[i] and new_end >= end[i] -> CLASH
return no clash
```

Two real defects in the clash rule

First, the day test is inside the wrong loop and re-derived every iteration. The code reads:

```
1 for i in range(len(start_timings)):
2     days_list = self.all_queries.get_days_list(days)           # candidate
3     previous_days_list = self.all_queries.get_days_list(all_days[i]) # held
4     for day in range(len(days_list)):
5         if days_list[day] in previous_days_list:
6             ... three overlap tests ...
```

`days_list` does not depend on `i`, so it is recomputed on every iteration of the outer loop. Harmless, wasteful, and a sign the loop grew rather than being designed. More notably, the three overlap tests sit inside the *inner* day loop, so for a candidate on MoWe clashing with a held course on MoWe, they run twice and return on the first. Correct, but only by accident of the early return.

Second, and this one is a genuine bug: touching intervals are treated as a clash. All three tests use `<=`. A candidate starting at exactly 12:45 against a held course ending at exactly 12:45 satisfies `start[i] <= new_start <= end[i]` and is rejected. Look at the real catalogue: 201/L1 runs 11:30 to 12:45 on MoWe, and back-to-back classes are the normal case in a university timetable. This rule forbids them. A strict `<` on the boundary comparisons is the fix. This is easy to demonstrate live and worth knowing before someone else demonstrates it to you.

The clash rule is only ever consulted by the UI

`AllQueries.add_course` does not call the clash checker. It is called only from `AddCoursesWindow.can_the_course_be_added` and `SwapCoursesWindow.can_the_course_be_added`, which are two separate, near-identical copies of the same 40 lines in two different files. So: a clash-free timetable is a property the *windows* maintain, not a property the data layer or the schema guarantees, and the rule is duplicated so it can drift. It has already begun to: the two copies call *different* query methods (the add path uses the unfiltered one that includes the cart; the swap path uses the filtered one with the exclusion arguments).

4.4 Rule 3: capacity, and what `current_capacity` actually means

`current_capacity` counts *empty seats*, not filled ones

This is the single most counter-intuitive thing in the codebase, and the column name actively works against you. Read `update_current_capacity`: enrolling calls it with "Subtract"; dropping calls it with "Add". So the number goes *down* as students join. It is **remaining seats**.

The shipped data confirms it: untouched sections read `current_capacity = 50`, equal to `max_capacity`, and the sections with enrollments read 47. A name like `seats_remaining` would have removed all doubt.

```

1 def update_current_capacity(self, course_id, section, add_or_subtract):
2     course = session.query(CourseList).filter_by(course_list_id=course_id,
3                                                    section=section).first()
4     if add_or_subtract == "Add":
5         course.current_capacity += 1
6     elif add_or_subtract == "Subtract":
7         course.current_capacity -= 1
8
9     if course.current_capacity == 0:
10        course.status = False
11    elif course.current_capacity > 0:
12        course.status = True
13    else:
14        print("Error: current_capacity is negative")
15        return False
16    ...

```

Listing 5: `queries.py`, the whole capacity rule.

So `status` is a derived flag: false ("Closed") exactly when no seats remain. The catalogue screens hide any section whose `status` is false, which is how a full section disappears from the list.

This was verified by running the real method against a copy of the database:

```

1 201/L2 before:                max=50, current=50, status=True
2 after an ENROLL (Subtract):    max=50, current=49, status=True
3 after two DROPS (Add, Add):    max=50, current=51, status=True  <-- exceeds max

```

Listing 6: Executed against a copy of `student.db`. Output is real.

`max_capacity` is never read by a single line of code

Verified by `grep` across every `.py` and `.ui`: the only occurrence of `max_capacity` anywhere in the project is the column declaration itself. It is written into the seed data and never consulted again. The ceiling on a section is not enforced; only the floor is (`current_capacity`

`== 0` closes it).

The consequence is not hypothetical. **The shipped database already contains two sections that are over capacity:** 330/L3 and 310/L2 both have `current_capacity = 51` against `max_capacity = 50`. Somebody dropped a course they had never enrolled in (or the seat count was decremented once and incremented twice), and nothing stopped the counter climbing past the maximum. Those rows are in the file you would demo from.

The fix is two lines in the same function that already has the “negative” branch: refuse to increment past `max_capacity`, and refuse to decrement below zero *before* committing rather than after. Note the existing negative branch returns `False` only *after* having already mutated the object, and its caller in `enroll.py` shows “Invalid capacity” and returns, leaving the session holding a dirty object. It happens not to be committed on that path, but nothing rolls it back either.

Capacity is not decremented where you would expect

Adding to the cart deliberately does *not* take a seat: the `update_current_capacity` call in `add_courses.py` is present but commented out. Only `Enroll` takes the seat. This is the right call (a cart should not hold inventory), but it also means there is **no reservation**: two students can both pass the “Open” check, both fill their carts with the last seat, and both enroll. The second one drives `current_capacity` to `-1`, at which point the negative branch fires, `status` is left stale, and the user sees “Invalid capacity”. For a single-user coursework demo this never happens. It is the honest answer to “what breaks if two students use this at once”, and the answer is: the seat count, immediately, because there is no locking and no constraint.

4.5 Rule 4: the swap

A swap moves the student from one section of a course to another section of the *same* course. The UI enforces exactly that:

```
1  # if section and course_id same then show messagebox and return
2  if course_id != course_id_enrolled:
3      QMessageBox.warning(self, "Error", "Cannot swap different courses.")
4      return
5
6  if course_id == course_id_enrolled and section == section_enrolled:
7      QMessageBox.warning(self, "Error", "Cannot swap the same course.")
8      return
```

Listing 7: `swap_courses.py`. Note the comment describes the opposite of the code.

So the two rules are: the courses must match, and the sections must differ. The Select button already filters the second table to sections of the chosen course, via `get_all_courses_by_std_id_and_course_id`, so the first rule mostly guards against the user clicking around it.

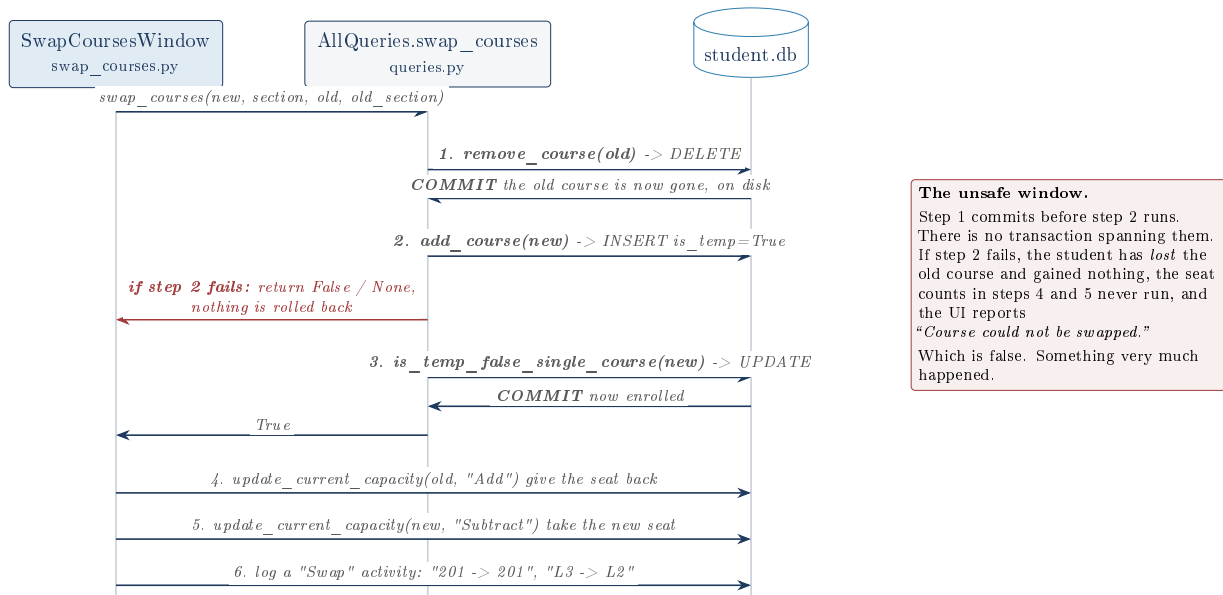


Figure 7: The swap, step by step, showing where the atomicity is missing. Verified by execution, below.

Jargon: Atomicity and transactions

An operation is **atomic** if it either happens completely or not at all, with no observable in-between. Databases provide this with a **transaction**: you begin one, make several changes, and **COMMIT** them as a single indivisible act, or **ROLLBACK** to undo the lot. A swap is the textbook case: “delete the old, insert the new” must be atomic, or a crash between the two loses the student’s course. This is the ‘A’ in ACID, and it is a core Databases-course concept, which makes it the most important finding in this document.

The swap is not atomic, and it destroys data. Verified by running it.

`AllQueries.swap_courses` calls `remove_course()` first, which does `session.delete(course)` followed by its own `session.commit()`. That commit is unconditional and it is outside the `try` block that guards the add. Only then does it attempt `add_course()`.

This was tested directly against a copy of the shipped database, by asking for a swap onto a section that does not exist so that step 2 must fail:

```

1 enrolled rows for student 20180 before:
2   [(201,'L1'), (202,'S2'), (205,'L3'), (330,'L3'), (310,'L2')]
3
4 swap_courses(new=999/'ZZ' (nonexistent), old=201/'L1') -> returned: None
5
6 enrolled rows for student 20180 after:
7   [(202,'S2'), (205,'L3'), (330,'L3'), (310,'L2')]
8
9 LOST:   [(201, 'L1')]
10 GAINED: []
  
```

Listing 8: Executed against a copy of `student.db`. Output is real.

The student’s enrollment in 201/L1 was deleted and committed, the replacement was never created, and the function returned `None`. `None` is falsy, so the caller in `swap_courses.py` takes the `if not can_be_added:` branch and shows the user “Course could not be swapped.” The user is told nothing happened. They have in fact just lost a course, permanently, and the seat is not even returned to the pool because steps 4 and 5 are below the early return.

Note also the bare `except:` inside `swap_courses` ends in `return` with no value, which is why the function returns `None` rather than `False`. Two different falsy failure signals from one

function.

The fix is small and you should be able to state it instantly: wrap steps 1 to 3 in one transaction. Do not commit inside `remove_course` when it is called from a swap; do the delete, do the insert, and commit once, with `session.rollback()` in the except. SQLAlchemy gives you this for free with `with session.begin():`. The reason it was not done is visible in the code: `remove_course` and `add_course` were each written to be self-contained, committing operations for their own screens, and `swap_courses` reuses them as-is. Composing two atomic operations does not give you an atomic operation. That sentence is the whole lesson. And this is what the dead `swap_id` / `remove_id` columns were for. The original design was going to *link* the new row to the old one rather than delete and re-insert. That design would not have had this bug.

4.6 Rule 5: the drop

The simplest path, in `drop_courses.py`: confirm with a Yes/No dialog, delete the `course` row, return the seat with `update_current_capacity(..., "Add")`, remove the table row, log a "Drop" activity. There is no rule about *whether* you may drop; you always may.

The drop screen's column headers do not line up with its data

`setup_drop_courses_table` sets nine headers: ["Course ID", "Course Name", "Section", "Instructor", "Days", "Start Time", "End Time", "Room", "Status"]. Then it fills only *eight* columns, because it merges start and end into one:

col	header says	the code actually puts there	
4	Days	days	ok
5	Start Time	"11:30:00 - 12:45:00"	both times in one cell
6	End Time	room	wrong
7	Room	status	wrong
8	Status	nothing	empty column

So on the drop screen the room number appears under a heading that says "End Time", the word "Open" appears under "Room", and the last column is blank. This is a visible defect on a screen a demo will certainly open. The functional part is unaffected, since `drop_course` reads columns 0 and 2, which are still correct.

4.7 The enroll screen, and a bug that cancels itself out

`enroll.py` is the checkout. It lists everything in the cart (`get_all_courses_by_id_and_type("Add")`), refuses if any row reads "Closed", then flips the whole cart to enrolled, then decrements a seat for each row.

```

1  for row in range(self.schedule_table_widget.rowCount()):
2      if self.schedule_table_widget.item(row, 8).text() == "Closed":
3          QMessageBox.about(self, "Enroll", "You cannot enroll in a closed course")
4          return
5
6  self.all_queries.is_temp_false_remove_type_add("Add")      # <-- commits EVERYTHING
7  QMessageBox.about(self, "Enroll", "You have successfully enrolled ...")
8
9  course_ids = []
10 for row in range(self.schedule_table_widget.rowCount()):
11     if self.schedule_table_widget.item(row, 0).checkState(): # <-- always True
12         course_ids.append(self.schedule_table_widget.item(row, 0).text())

```

Listing 9: `enroll.py`, `enroll_action`. The order matters.

`if item.checkState():` is always true in PyQt6. Verified.

The intent is clearly “only the rows the user ticked”. It does not work, and the reason is a genuine PyQt5-to-PyQt6 migration trap.

In PyQt5, `checkState()` returned a plain integer, and `Qt.Unchecked` was 0, so `if item.checkState():` was false for an unticked row. PyQt6 replaced those integer constants with real Python enums, and **a Python enum member is truthy regardless of its value** unless the enum defines otherwise. `Qt.CheckState` is a plain `enum.Enum`, not an `IntEnum`.

Verified by installing PyQt6 6.11 in a throwaway environment and asking:

```
1 >>> repr(Qt.CheckState.Unchecked)
2 <CheckState.Unchecked: 0>
3 >>> bool(Qt.CheckState.Unchecked)
4 True
```

So the condition is true for every row, ticked or not. The whole cart is always collected.

Now the interesting part: **this bug has no visible effect**, for two compounding reasons. First, `is_temp_false_remove_type_add("Add")` on the line above has already enrolled the entire cart in the database before the loop runs; the loop only decides what gets logged and whose seat gets taken. Second, no code ever calls `setCheckState` on these items and the table is set to `NoEditTriggers`, so no checkbox is ever shown and the user could not tick anything anyway. The intended feature does not exist in the UI, and the code that would have implemented it is broken in a way that happens to produce the behaviour the rest of the screen assumes: enroll everything.

Two wrongs make a right here. It is a much better story than a bug: it shows you can trace a defect to the exact language-level semantics that caused it, and then judge honestly that it does not currently matter. What it *does* mean is that the dead code should be deleted, because the day someone adds real checkboxes, this silently enrolls the unticked ones too.

The success dialog fires before the work that can fail

“You have successfully enrolled in the selected courses” is shown immediately after the `is_temp` flip and *before* the capacity loop. If `update_current_capacity` then returns false, the user gets “Invalid capacity” straight after being congratulated. This is the same pattern the attendance module deliberately avoids (Section 7.1), which makes the inconsistency worth noting: the newer code learned the lesson, the older code did not.

5 The data-access layer

`AllQueries` in `queries.py` is 410 lines and about 25 methods, and it is the project’s best structural idea: no window ever writes a query, so the screens agree on a contract. The README says this was deliberate, to keep a two-person team’s merges tractable, and the code bears that out.

5.1 `glob_id`: how the layer knows who you are

Every method that touches student-specific data begins the same way:

```
1 def get_all_courses_by_id(self):
2     std_id = self.glob_id
3     ...
```

`glob_id` is set in `__init__` to a hardcoded 20180 and then overwritten by every screen with `self.all_queries.update_std_id(self.all_queries.get_std_id_by_email(self.email))`.

The README overstates this one, and the truth is more interesting

The README's "What I learned" says: "*AllQueries.glob_id (defaulting to a hardcoded 20180) is set by whichever window ran last, so the query layer's correctness depends on call order.*"

Read the code: `glob_id` is assigned in `__init__`, so it is an **instance** attribute, not a class attribute. Every screen constructs its *own* `AllQueries()` and immediately sets its own `glob_id` from its own `self.email`. So screens do not clobber each other's student id, and the stated call-order hazard does not exist as described. Correct the README rather than repeat this claim in an interview: the fix is one word, "instance", and being the person who caught it is better than being the person who recited it.

The real hazards are different and both worth defending:

- 1. The default 20180 is a live student.** It is Alice, a real row in the shipped database. Any `AllQueries()` constructed without a subsequent `update_std_id` silently operates *as Alice*. Verified: constructing `AllQueries()` and reading `glob_id` gives 20180 with no login of any kind. Every screen's `__init__` also defaults `email` to `alice20180@st.habib.edu.pk` when none is passed, which is what the `if __name__ == "__main__"` block at the bottom of every module does. So each screen is independently runnable as Alice, with no password and no verification code. That is the actual security-shaped hole, and it is a debugging convenience that shipped.
- 2. The session genuinely is shared.** `session` is a module-global in `student_database.py`, created once at import. Every `AllQueries` instance uses the same one. So the thing the README worried about is real, it is just one level down: not the id, the session. That is what makes the swap's partial commit in Section 4.5 durable.

5.2 Error handling, uniformly

Nearly every mutating method follows one shape:

```

1 session.add(new_course)
2 try:
3     session.commit()
4 except:
5     print("Error: " + str(sys.exc_info()[0]))
6     print(traceback.format_exc())
7     session.rollback()
8     return False
9 return True

```

Bare except: everywhere, and print as the logging strategy

`except:` with no exception class catches *everything*, including `KeyboardInterrupt` and `SystemExit`. It should be `except Exception:` at minimum, and really `except SQLAlchemyError:`. The failure is reported by printing to a terminal that a GUI user is not looking at, and returning `False`, which some callers check and some ignore.

The consistency is genuinely a plus: one shape, applied everywhere, is easier to fix than twenty bespoke ones. Replacing `print` with the `logging` module and the bare `except` with a typed one is a mechanical change across about a dozen methods. The README already lists this as a follow-up.

remove_course does not check that the row exists

```

1 course = session.query(Course).filter_by(...).first()
2 session.delete(course)      # <-- course may be None
3 try:
4     session.commit()

```

The `delete` is *outside* the `try`. If the row is not found, `course` is `None` and `session.delete(None)` raises `UnmappedInstanceError`, which propagates past the `try` that was supposed to guard it, out of `remove_course`, out of `swap_courses`, and into the window's own bare `except`, where it becomes a printed traceback and a silent return. Two of the three callers happen to pass ids read out of a table populated from the database, so the row does exist. It is guarded by luck.

5.3 Two methods that do the same job

Two methods insert an `Activity` row:

```

add_data_to_activity_log(std_id, activity_type, course_id, section)
insert_activity(std_id, activity_type, description, date_val, time_val)

```

There is also a third, `insert_attendance_activity`, which formats a string and inserts an `Activity` row.

Three activity-insert methods; one is dead

`insert_attendance_activity` is never called by anything. Verified by `grep`: `attendance.py` calls `insert_activity`. The section comment above it in `queries.py` announces `# ----- NEW METHOD FOR ATTENDANCE -----`, so it is the first attempt, superseded by `insert_activity` directly below it, and never removed. Its docstring even contains a smart apostrophe, marking it as pasted from somewhere else.

The difference between the two survivors is that `add_data_to_activity_log` stamps the time itself with `datetime.now()`, while `insert_activity` takes date and time as *strings* and parses them with `strptime`, falling back to `now()` if parsing fails. The caller in `attendance.py` formats `now()` into strings, passes them, and has them parsed back. That is a round trip through text for no reason. One method taking a `datetime` would have replaced all three.

Also note `add_data_to_activity_log` contains a query whose result is discarded:

```

1 course = None
2 if course_id is not None:
3     course = session.query(CourseList).filter_by(course_list_id=course_id).first()
4 # `course` is never used again.

```

A wasted `SELECT` on every logged action.

5.4 The destructive `__main__` block**Running python queries.py wipes your data**

```

1 if __name__ == "__main__":
2     all_queries = AllQueries()
3     all_queries.delete_all_data_activity_log()
4     all_queries.delete_everything_in_course_table()
5     print(all_queries.get_current_day())

```

That deletes **every row** of activity and **every row** of course, for every student, with

no confirmation, as the price of running the file. It is obviously a development reset script that was left in the shipped module. It should be a separate `reset_db.py`, or at minimum behind an `argv` check. Note also that `delete_all_data_activity_log` and `delete_everything_in_course_table` ignore `glob_id` entirely: they are not scoped to a student despite living on an object whose entire purpose is to scope queries to a student.

6 Login and the verification flow

This is one of the areas the README records as Salman's, and it is the most carefully written code in the project.

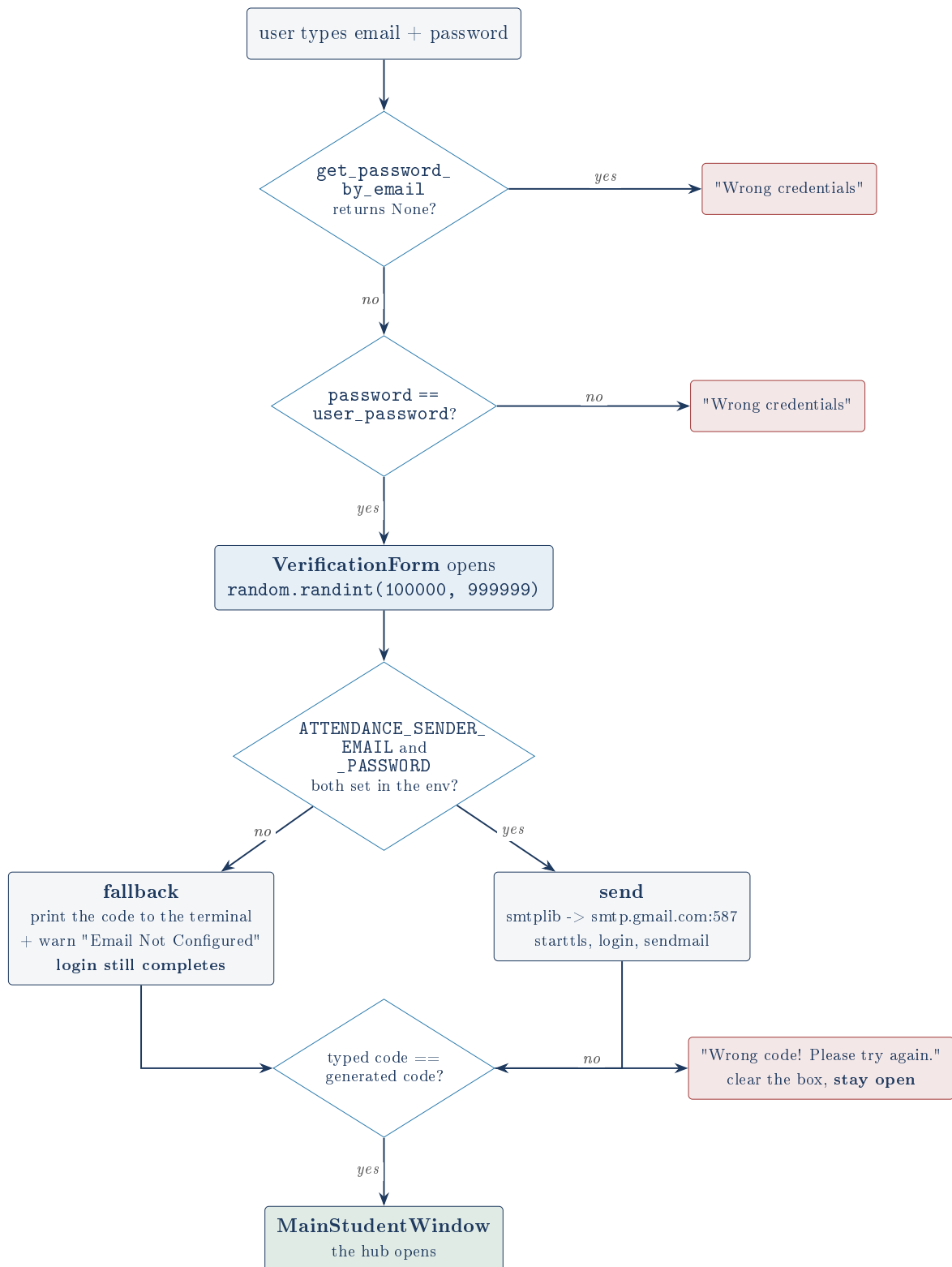


Figure 8: Login and multi-factor verification, including the no-credentials fallback that lets anyone who clones the repository still log in.

Jargon: MFA, SMTP, and an App Password

MFA (Multi-Factor Authentication) means proving who you are with more than one kind of evidence: something you know (the password) plus something you have (access to the mailbox the code was sent to). **SMTP** (Simple Mail Transfer Protocol) is the protocol for

sending mail; Python's `smtplib` speaks it. Gmail will not accept your normal password from a script, so you generate an **App Password**, a separate 16-character secret scoped to one application, which is what `ATTENDANCE_SENDER_PASSWORD` is meant to hold.

The best decision in the project: gate the network call, then tell the truth

Anyone cloning this repository has no Gmail App Password. A blind `smtplib.SMTP(...)` would hang on a connect or throw, and the login would be unusable. So both email paths check first:

```
1 sender_email = os.environ.get("ATTENDANCE_SENDER_EMAIL", "")
2 sender_password = os.environ.get("ATTENDANCE_SENDER_PASSWORD", "")
3 if not sender_email or not sender_password:
4     ...
```

and then each picks a *truthful* fallback for its own context. `verification.py` prints the code to the terminal and says so in a dialog, so login completes. `attendance.py` returns `False` and the caller says “confirmation emails could not be sent” rather than claiming they were. That last distinction is the whole point. The lazy version returns `True` and shows “emails sent!”. This code refuses to lie to the user about something it did not do. Two env vars drive both paths and `.env.example` documents them.

Real weaknesses in the verification flow, stated plainly

- **The code is held in memory on the client and compared there.** `self.verification_code` is a string attribute on the window; `handle_verification` compares the typed text to it. In a desktop app the user owns the process, so this is theatre against a determined user. It is a faithful demonstration of the MFA *flow*, not a security control. Say that rather than overselling it.
- **random.randint is not cryptographic.** `random` is a Mersenne Twister seeded predictably; `secrets.randbelow` is the right call for anything that is supposed to be unguessable. Given the previous point, it changes nothing here, but it is the correct answer to “how would you generate that code”.
- **No expiry, no attempt limit.** The code lives as long as the window and can be retried without limit. A real MFA has a TTL and a lockout.
- **The env vars are misnamed for one of their two jobs.** They are called `ATTENDANCE_SENDER_*` and are also used for login verification, which is not attendance. The README and `.env.example` are consistent about it, so it is a naming debt rather than a trap, but a reviewer will ask.
- **Failure to send closes the window.** In `send_verification_email`, the except shows an error and calls `self.close()`. But `VerificationForm` was constructed with `super().__init__(login_parent)`, and the login window was hidden by `handle_login`. Reading the code, a send failure leaves the verification window closed and the login window still hidden, with no path back. **Inferred**, not verified: the app could not be run to confirm what the user sees at that moment.

The show/hide password button is inverted

```
1 self.password_visible = False
2 self.toggle_password_visibility()      # called once at construction
3
4 def toggle_password_visibility(self):
```

```

5     self.password_visible = not self.password_visible
6     if self.password_visible:
7         self.show_hide_button.setText("Show")
8         self.password_textbox.setEchoMode(QLineEdit.EchoMode.Password)    # HIDDEN
9     else:
10        self.show_hide_button.setText("Hide")
11        self.password_textbox.setEchoMode(QLineEdit.EchoMode.Normal)      # VISIBLE

```

The branch where `password_visible` is `True` is the branch that *hides* the password. The flag means the opposite of its name. The labels happen to read correctly if you interpret them as “click me to Show”, and the constructor’s warm-up call happens to leave the box masked at startup, so the user-visible behaviour is right. The code says the opposite of what it does. Rename the flag to `password_hidden` and it reads correctly.

7 The remaining screens

7.1 Attendance

Attendance has no table, and nothing it shows is real

`setup_attendance_table` contains this, and this is the entire data source:

```

1 dummy_data = [
2     ("2025-03-01", "Monday", "Present"),
3     ("2025-03-02", "Tuesday", "Absent"),
4     ("2025-03-03", "Wednesday", "Present"),
5     ("2025-03-04", "Thursday", "Absent")
6 ]

```

Four hardcoded rows, four fixed dates in March 2025, no student id involved. There is no **Attendance** model and no attendance table in the schema; verified against `student.db`, which has exactly five tables and none of them is attendance.

The consequence is that marking attendance does not persist. Reopen the screen and you are back to Present/Absent/Present/Absent regardless of what you set. The README states this in “What I’d do differently” and it is the right call to state it.

What *does* persist is the side effect: every press of Mark Attendance writes one **Activity** row per table row, unconditionally, whether or not the value changed. So four rows per press, forever. The shipped log’s 116 rows include several of these. The docstring on `handle_mark_attendance` even says “check if the user changed the status from the original dummy data”, and then does not: it re-sends and re-logs all four every time.

The email handling here is the good part, and it is the mirror image of the verification fallback: `send_attendance_email` returns a boolean, `handle_mark_attendance` accumulates `all_emails_sent` across the rows, and the final dialog says either “Attendance marked and emails sent!” or “Attendance marked, but confirmation emails could not be sent” with the env-var names in the message. That is an honest status, and it is the pattern `enroll.py` should have copied.

7.2 View schedule and the Excel export

Two modes on one table, switched by `self.filter_mode`:

- **Day** (the default): shows column 0, “Current Day”, and includes only the courses that meet today. The filter is `if current_day[:2] not in course_days_list: continue`, that is, take the first two characters of `datetime.now().strftime("%A")` (so “Monday” becomes “Mo”) and

look for it among the section’s two-character day codes. This is the same encoding as the clash detector, and it is why the codes must be exactly two characters.

- **Week:** hides column 0 and lists every enrolled course once.

Day mode is fragile in a way that only shows up on the wrong day

`current_day[:2]` maps Monday to “Mo”, Tuesday to “Tu”, Wednesday to “We”, Thursday to “Th”, Friday to “Fr”, Saturday to “Sa”, Sunday to “Su”. That works for this catalogue. It relies on the process locale producing English day names: on a machine with a non-English locale, `strftime("%A")` returns a translated name and the day view silently shows nothing at all. No error, just an empty table. It also means the default view is empty on any day the student has no classes, which looks identical to a failure.

The export is genuinely thoughtful:

```
1 def save_data(self, excel_file_path, df):
2     while not is_file_found:
3         try:
4             pandas.read_excel(excel_file_path)          # if this succeeds, it exists
5             counter += 1
6             excel_file_path = ... + f'/{name} ({counter}).xlsx'
7         except FileNotFoundError:
8             df.to_excel(excel_file_path, index=False)
9             is_file_found = True
```

Listing 10: `view_schedule.py`: do not clobber an existing export.

Jargon: `pandas`, `DataFrame`, `openpyxl`

pandas is the standard Python library for tabular data. Its central object is the **DataFrame**, a table with named columns. `DataFrame.to_excel` writes it out as a real `.xlsx` spreadsheet, delegating the file format to **openpyxl**, which is why both are in `requirements.txt` even though no line of code imports `openpyxl` directly.

Detecting a file by trying to parse it

Probing with `read_excel` and catching `FileNotFoundError` works, and the auto-increment to `schedule_by_week(1).xlsx` is a nice touch that the README is right to call out. But it reads and fully parses the whole spreadsheet just to learn whether it exists, when `os.path.exists(path)` answers the same question without opening anything. Worse, it only catches `FileNotFoundError`: if the existing file is corrupt, or locked because the user has it open in Excel, `read_excel` raises something else, which is unhandled, and the export dies with a traceback in a terminal nobody is reading.

Day-mode export is disabled, and the dead check gives away why

`export_schedule` begins with `if self.filter_mode == "Day":` and returns “Export Not Available”. Then, forty lines later, it asks `if self.filter_mode == "Day":` again, and that second block is unreachable. The README explains the history: the day view hides a column, the export was writing misaligned data, so it was disabled rather than shipped broken. That is the right decision. The residue is the dead second check and a `columns` list assigned in the first branch and never used.

The consequence worth knowing for a demo: `filter_mode` starts at “Day”, so **Export does nothing until you press Week first**. That looks like a broken button.

The schedule window logs an activity in its constructor

`__init__` writes a "Filter by Day" activity row before the user has done anything, simply because the screen opened in day mode. So the log conflates "opened the schedule" with "pressed the Day button". Combined with the attendance screen's four rows per press, this is why the shipped log has 116 rows.

7.3 Activity log

The simplest screen: read every `Activity` row for the student, render six columns. It is the only place the log is consumed.

A guard that hides data

```
1 if activity_log_data.course_id is not None and activity_log_data.course_section is not
   None:
2     self.activity_label_table_widget.setItem(row, 2, QTableWidgetItem(str(...course_id)))
3     self.activity_label_table_widget.setItem(row, 3, QTableWidgetItem(...course_section))
```

Column 2 (Course ID) is populated only if *both* the id and the section are non-null. So a log entry with a course id but no section shows a blank Course ID column, even though the id is right there in the row. Column 3 has no such guard and is set unconditionally, which for a null section passes `None` into `QTableWidgetItem`. There is no sorting and no pagination: 116 rows today, and it grows without limit because nothing ever prunes it.

7.4 The window-transition machinery

Every screen defines `closed = pyqtSignal()` and emits it from `closeEvent`. The hub connects that signal to its own `reopen_login` (which just calls `self.show()`), and hides itself when opening a child. That is how closing a child brings the hub back.

Jargon: Signals and slots

Qt's event mechanism. An object **emits** a **signal** ("I was closed") without knowing or caring who is listening. Any number of functions, called **slots**, can be **connected** to it and are called when it fires. It is Qt's version of the observer pattern, and it is what decouples the child windows from the hub.

`closeEvent` never calls `super()`, and the hub calls it by hand

Every `closeEvent` override in the project has `# super().closeEvent(event)` commented out and never accepts or ignores the event. Qt's default is to accept, so the windows do close, but the override is doing something other than what its signature promises.

`MainStudentWindow.logout` is stranger still: it sets a flag and then calls `self.closeEvent(event=None)` *directly*, invoking an event handler as an ordinary method with a fabricated null event. It works only because the body never touches `event`. The intended API is `self.close()`, which lets Qt deliver a real event.

There is also a straightforward duplicate in `MainStudentWindow.activity_log`:

```
1 self.activity_log_window = ActivityLogWindow()
2 self.activity_log_window = ActivityLogWindow(email=self.email)
```

Two windows are constructed; the first is immediately orphaned by the reassignment. It is never shown, but its constructor ran, which means it opened the `.ui` file, ran `get_std_id_by_email` for the *default* student (Alice), and queried her activity log. A wasted

window and a wasted round of queries on every click.

7.5 The name collision

Two different classes are both called `ViewScheduleWindow`

`enroll.py` defines class `ViewScheduleWindow(QMainWindow)`, which loads `enroll.ui` and is the *checkout* screen. `view_schedule.py` defines class `ViewScheduleWindow(QMainWindow)`, which loads `view_schedule.ui` and is the *schedule* screen. Same name, different files, different purposes.

Nothing breaks, because `main_window.py` imports the *schedule* one and `add_courses.py` imports the *enroll* one, and no module imports both. It is a trap lying in wait for the first person who needs both in one file, and it makes the codebase hard to talk about. The history is visible: `enroll.py` is the older file (its class was clearly copied from the *schedule* screen and repurposed), and the `.ui` it loads is still named `view_schedule_window` internally, as is `add_courses.ui`. Three files carry the fingerprint of one original. The README's final "Challenges" bullet already records this collision, which is the right instinct.

An orphan interface file

`add_remove_interaction.ui` is 8.5 KB, the largest `.ui` in the project, and no Python file loads it. Verified by `grep`. It is a designed screen that was never wired up.

8 Packaging and running

Jargon: Docker, image, container

Docker packages an application together with its operating-system dependencies into an **image**, a frozen filesystem plus a start command. Running an image gives you a **container**, an isolated process that behaves the same on any machine with Docker. It is the standard answer to "it works on my machine".

The `Dockerfile` is short and its shape is right: start from `python:3.10-slim`, install the system libraries Qt needs, set `WORKDIR /app` (which is what makes the relative `loadUi` paths resolve), install `requirements.txt` *before* copying the code so that Docker's layer cache is not invalidated by every edit, copy everything including the seeded `student.db`, set `QT_QPA_PLATFORM=xcb`, and run `login_page.py`.

Jargon: X11 forwarding, and why a GUI in a container is awkward

A container has no screen. On Linux the display is managed by a server called **X11** which listens on a socket at `/tmp/.X11-unix`. The run scripts mount that socket into the container and pass the `DISPLAY` variable, so the containerised app draws on *your* desktop. `xhost +local:docker` is the command that permits it, and it is a blunt instrument: it grants any local container access to your display for the session.

Docker can never send an email, by construction

Verified by `grep`: neither the `Dockerfile` nor any of the three run scripts mentions `ATTENDANCE_SENDER_EMAIL` or `ATTENDANCE_SENDER_PASSWORD`. No `ENV`, no `-e` flag, no `--env-file`. A container gets a fresh environment, so those variables are always empty inside it.

The app therefore *always* takes the fallback path in Docker: the verification code is printed to the container’s terminal and attendance always reports that emails could not be sent. Because the fallback is honest, this degrades gracefully instead of breaking, which is a real vindication of the design. But the README’s claim that the scripts “forward the host display into the container” is the whole story only for the display. If you demo from Docker, email is off, and adding `-e ATTENDANCE_SENDER_EMAIL -e ATTENDANCE_SENDER_PASSWORD` to the `docker run` lines is a one-line fix per script.

The Dockerfile probably does not build any more. Labelled inferred.

The `apt` line installs `libgl1-mesa-glx`. The `python:3.10-slim` tag today is built on Debian 12 (bookworm), and `libgl1-mesa-glx` was **removed** in bookworm, replaced by `libgl1` and `libglx-mesa0`. If that is so, `apt-get install` exits non-zero and the image build fails at step 2.

This is inferred and was not verified. The Docker daemon was not running on the machine this document was written on, so no build was attempted. Treat it as the first thing to check before demoing from Docker: run `docker build -t pscs-app .` and see. If it fails, change `libgl1-mesa-glx` to `libgl1` and it should proceed. Note this is a bit-rot issue, not a mistake: the line was almost certainly correct against the base image of the day.

The run scripts contain characters that are not plain ASCII

All three (`run-linux.sh`, `run-macos.sh`, `run-windows.ps1`) contain a hammer emoji in their build message and a non-breaking hyphen (Unicode U+2011, not the ordinary U+002D) in the phrase “Auto-build” in their header comment. Both are in comments and echo strings, so they are harmless, but they are the fingerprint of text pasted from a rich-text editor. The non-breaking hyphen in particular is invisible in most editors and is the sort of thing that breaks a naive `grep`. (Those exact bytes are described here rather than reproduced, because this document is typeset with a font stack that has no glyph for either one.)

`queries.py` and `verification.py` carry the same fingerprint: smart apostrophes inside docstrings, where an ASCII apostrophe was meant.

8.1 Dependencies, and what each is for

package	pinned	role
PyQt6	>=6.6,<7	every window, widget, dialog and signal
SQLAlchemy	>=2.0,<3	the ORM; models and every query
pandas	>=2.0	builds the DataFrame for the Excel export
openpyxl	>=3.1	the <code>.xlsx</code> writer pandas delegates to

The upper bounds on the first two are correct practice: a major version bump of either is a breaking change. `pandas` and `openpyxl` have no upper bound, which is a minor inconsistency. The whole dependency set exists to serve one feature each, and there is nothing gratuitous in it.

9 The verification story, and what it does not cover

The README describes a test methodology that no file in the repo implements

The README's Challenges and "What I learned" describe, in specific detail, a headless smoke test: setting `QT_QPA_PLATFORM=offscreen`, monkeypatching the static `QMessageBox` methods (`information`, `warning`, `critical`) so modals record their calls instead of blocking, and driving login through to the main window plus every sub-screen in one pass. It says "This is how the project was last verified."

There is **no test file in the repository**. `git ls-files` lists no `tests/` directory, no `test_*.py`, no pytest configuration, and no CI workflow. `.gitignore` mentions `.pytest_cache/`, which is boilerplate.

The most likely reading, and this is **inferred**, is that the smoke test was written as a throw-away script during a verification pass and never committed. That is a perfectly honest thing to have done, but the README's present tense reads as though the harness is part of the project, and an interviewer who goes looking for it will not find it. Two options, both fine: commit the script, or reword the README to say the test was run once, ad hoc, and not kept. The README's own last bullet already concedes "Add tests. Everything here was verified by clicking through the app", which sits oddly beside the detailed description of the harness. The two passages should be reconciled.

What this document verified, and what remains open

Verified by execution: the capacity arithmetic and its overflow past `max_capacity`; the swap's data loss; the foreign keys' invalidity under `PRAGMA foreign_keys = ON`; `bool(Qt.CheckState.Unchecked) == True`; the `glob_id` default of 20180; the ORM relationship warning. **Verified by inspection of `student.db`:** every row count, the day encodings, the over-capacity rows, the polluted `activity.course_id` values, the all-NULL dead columns, the empty filter table. **Verified by `grep`:** `max_capacity` unread, `courses_database.py` unimported, `add_remove_interaction.ui` unloaded, `insert_attendance_activity` uncalled, the env vars unforwarded by Docker.

Open, and labelled inferred where claimed: anything about on-screen behaviour (PyQt6 was not present, and the app needs a desktop); the Dockerfile's `libgl1-mesa-glx` problem; the verification window's dead end on a send failure; the history behind the missing test file.

10 Findings, ranked

If you have five minutes before an interview, read only this.

1. **The swap loses data** (§4.5). Delete commits, then insert is attempted; a failure between them destroys the enrollment and the UI says nothing happened. Verified by running it. Fix: one transaction around both.
2. **The foreign keys are structurally invalid and never enforced** (§3.3). They reference one column of a composite primary key. SQLite's default of not checking is the only reason the app works. Verified.
3. **`max_capacity` is never read** (§4.4), and the shipped database already has two sections at 51 seats against a maximum of 50. Verified.
4. **Passwords are plaintext** and compared with `==`. Already documented in the README, which is the right way to carry it.

5. **The clash rule rejects back-to-back classes** (§4.3) because all three interval tests use `<=` on the boundary.
6. **Attendance is four hardcoded rows** with no table behind them (§7.1), and it writes four log rows per press regardless of change.
7. **The README is wrong in three places**: `courses_database.py` is not a seeding helper (it is a dead, incompatible `Student` model); `glob_id` is per-instance so the stated call-order hazard does not exist as described (the shared *session* is the real one); and the described headless test harness is not in the repository.
8. **Dead weight**: the `filter` table (0 rows, no code), the `swap_id` and `remove_id` columns (all `NULL`), `courses_database.py`, `add_remove_interaction.ui`, `insert_attendance_activity`, the double-construction in `activity_log()`, and the unreachable second day check in `export_schedule`.
9. `python queries.py` **wipes the activity and course tables** with no confirmation.
10. **Cosmetic but demo-visible**: the drop screen's headers are off by one from column 6; Export appears broken until Week is pressed; "Test Course" at 02:30 on MoSa is in the catalogue.

And what is genuinely good, which is worth saying too

The `AllQueries` choke point is a real architectural decision that paid off, and the README's account of why (a two-person team needed one contract between screens) is credible and supported by the code. The cart/enroll split is the right model. The environment-gated email with a *truthful* fallback in both paths is better engineering than most coursework contains, and it is the reason the app still runs for anyone who clones it. The Excel auto-increment shows someone thought about the second time the button is pressed. And disabling the day export rather than shipping a misaligned spreadsheet was the right call, made for the right reason.

Above all: the README already names most of the weaknesses in this document, in public, on a portfolio piece. That is the hardest thing on this list to do, and it is the thing to lead with.